

COURSE CODE

SBT-DF203

COURSE TITLE

Basic Networking Skills for Digital Forensics

LAB TITLE

SYN Flood Attack Investigation Using tshark

NAME

Kumpon Emmanuel Nandom

STUDENT ID

2025/FWSD/11325

DATE

11/09/2026

PLATFORM USED

Kali Linux

INSTRUCTOR

Mr. Aminu Idris, AMCPN
Commandant

Executive Summary

This report describes an authorized investigation of TCP SYN-flood-style traffic conducted on a local loopback interface (127.0.0.1) against an Apache service. This aligns with the requirements for SBT-DF203 Lab 3. I first captured a normal HTTP handshake to use as a baseline. Then, I sent a limited four-packet SYN probe using Scapy to mimic the packet pattern of a SYN flood, ensuring no actual denial-of-service occurred.

In my analysis, I found seven SYN packets from different, randomly chosen source ports. Each was met with a SYN-ACK response from the Apache service, but six of the seven half-open sessions were reset. Overall, there were no completed handshakes and no HTTP requests, compared to one completed session in the normal baseline. These findings fit the expected pattern of unacknowledged SYN traffic. However, the scale, rate, and duration of my test (7 packets in two short bursts) were much less than what would cause a real denial-of-service event. At no point did I see any impact on the Apache service.

Mini Evidence and Chain-of-Custody Worksheet

Case/lab identifier	SBT-DF203-Lab3-ponzhi
Trainee name	Kumpon Emmanuel Nandom
Date and time started	11 September 2026, 19:09
Evidence file name(s)	mySYNFloodCapture.pcap, normal_http.pcapng, bounded_syn_activity.pcapng
Source or generation method	I downloaded the file mySYNFloodCapture.pcap from the GitHub repository provided by ICDEFA using wget. The file normal_http.pcapng is a capture of HTTP traffic on tcp port 80 for 15 seconds using the tshark tool on the loopback interface while performing a curl request to the local Apache server. The file bounded_syn_activity.pcapng is a 20-packet capture during a 4-packet SYN probe allowed by Scapy against 127.0.0.1:80.
Original SHA-256	mySYNFloodCapture.pcap: 14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 bounded_syn_activity.pcapng: e5bbc6fd8689c505b0e521c2129b05a60893814adf14aa64e6a6c632b239dff0
Working-copy SHA-256	bounded_syn_activity_working.pcapng: e5bbc6fd8689c505b0e521c2129b05a60893814adf14aa64e6a6c632b239dff0 (identical to the original — working copy integrity confirmed)
Analysis workstation/VM	Kali Linux VM, hostname "kali", user "ponzhi"
Notes on any changes	Dumpcap showed "Permission denied" when trying to write directly into the evidence/ folder, even though we checked the folder's ownership,

permissions, ACLs, AppArmor status, and mount type, and everything seemed normal. To work around this issue, I captured data to /tmp and then moved the finished file into evidence/ using sudo mv. After that, I changed the ownership back with chown. This workaround only moves the completed capture file after tshark closes it and does not change the packet content, which is confirmed by the matching SHA-256 hash of the file.

Lab Folder Structure and Evidence Preparation

```
ponzhi@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help
(ponzhi@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}

(ponzhi@kali)-[~]
$ cd ~/SBT-DF203-Lab3

(ponzhi@kali)-[~/SBT-DF203-Lab3]
$ pwd
/home/ponzhi/SBT-DF203-Lab3

(ponzhi@kali)-[~/SBT-DF203-Lab3]
$ find . -maxdepth 1 -type d -print
.
./evidence
./working
./exported
./reports
./scripts
./screenshots
```

```
ponzhi@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help
(ponzhi@kali)-[~/SBT-DF203-Lab3]
$ sudo apt install -y apache2 tshark wireshark python3-scapy
apache2 is already the newest version (2.4.68-1).
tshark is already the newest version (4.6.6-1).
wireshark is already the newest version (4.6.6-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
python3-scapy set to manually installed.
The following packages were automatically installed and are no longer required:
  bluez-hcidump      libhdf5-310      libpoppler156    python3.13
  firmware-marvell-presteria libhdf5-hl-310  libpython3.13-dev python3.13-dev
  iucode-tool        libntfs-3g89t64 libvidstab1.1    python3.13-minimal
  libabsl20260107    libopenjph0.27  libx265-215      python3.13-venv
  libbluray3         libperl5.40     perl-modules-5.40
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 225

(ponzhi@kali)-[~/SBT-DF203-Lab3]
$ sudo systemctl enable --now apache2
```

```
ponzhi@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help
(ponzhi@kali)-[~/SBT-DF203-Lab3]
$ wget -O evidence/mySYNFloodCapture.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'
--2026-09-11 19:09:45-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.11.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 260716 (255K) [application/octet-stream]
Saving to: 'evidence/mySYNFloodCapture.pcap'

evidence/mySYNFloodCapt 100%[=====>] 254.61K 455KB/s in 0.6s

2026-09-11 19:09:47 (455 KB/s) - 'evidence/mySYNFloodCapture.pcap' saved [260716/260716]

(ponzhi@kali)-[~/SBT-DF203-Lab3]
$
```

```
ponzhi@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help
n/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.11.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 260716 (255K) [application/octet-stream]
Saving to: 'evidence/mySYNFloodCapture.pcap'

evidence/mySYNFloodCapt 100%[=====>] 254.61K 455KB/s in 0.6s

2026-09-11 19:09:47 (455 KB/s) - 'evidence/mySYNFloodCapture.pcap' saved [260716/260716]

(ponzhi@kali)-[~/SBT-DF203-Lab3]
$ sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 evidence/mySYNFloodCapture.pcap

(ponzhi@kali)-[~/SBT-DF203-Lab3]
$
```

Part A - Establish a Normal Handshake Baseline

A 15-second recording was taken on interface lo while sending a single curl request to <http://127.0.0.1/>. This shows what a standard HTTP session looks like at the packet level.

```
ponzhi@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help
mv /tmp/normal_http.pcapng evidence/normal_http.pcapng
ls -lh evidence/normal_http.pcapng
[1] 80297
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100  10703  100  10703    0      0  965.8k    0      0      0      0
10
[1] + done      sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w /tmp/normal_http.pcapng
mv: cannot open '/tmp/normal_http.pcapng' for reading: Permission denied
ls: cannot access 'evidence/normal_http.pcapng': No such file or directory

(ponzhi@kali)~-[~/SBT-DF203-Lab3]
$ sudo mv /tmp/normal_http.pcapng evidence/normal_http.pcapng

(ponzhi@kali)~-[~/SBT-DF203-Lab3]
$ sudo chown ponzhi:ponzhi evidence/normal_http.pcapng

(ponzhi@kali)~-[~/SBT-DF203-Lab3]
$ ls -lh evidence/normal_http.pcapng
-rw-r--r-- 1 ponzhi ponzhi 13K Sep 11 23:25 evidence/normal_http.pcapng

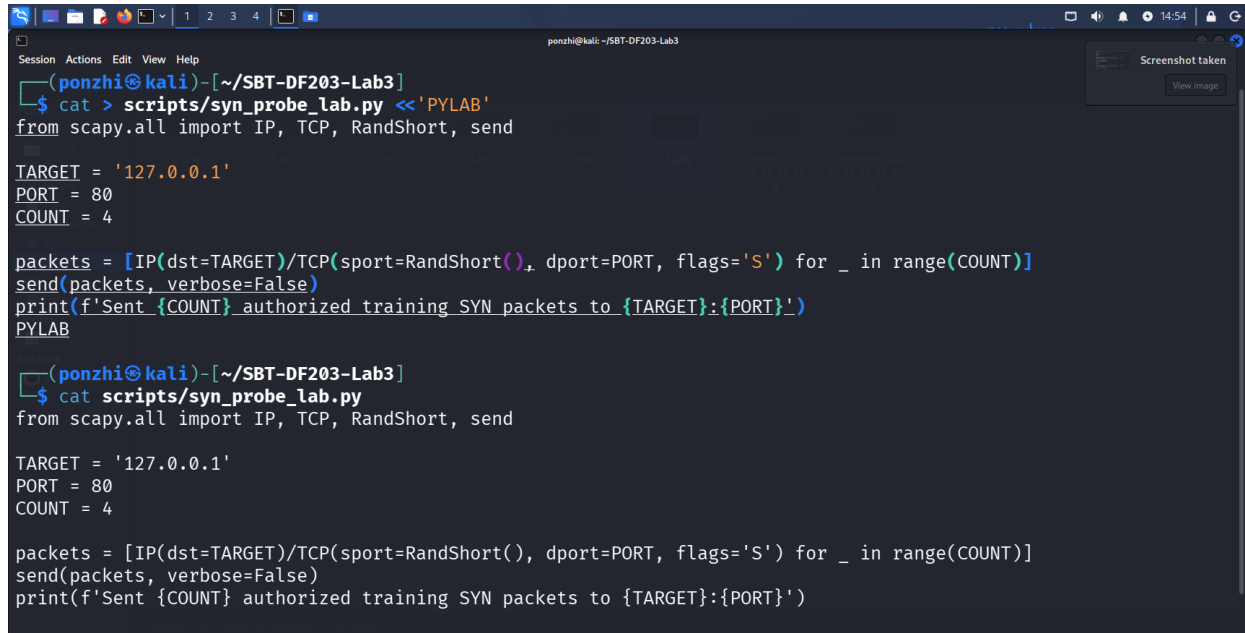
(ponzhi@kali)~-[~/SBT-DF203-Lab3]
$

(ponzhi@kali)~-[~/SBT-DF203-Lab3]
$ tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/normal_handshake_flags.tsv
1      2026-09-11T23:24:49.481452878+0100      127.0.0.1      51120      127.0.0.1      80      0x0002
2      2026-09-11T23:24:49.481471044+0100      127.0.0.1      80      127.0.0.1      51120      0x0012
8      2026-09-11T23:24:49.490317860+0100      127.0.0.1      51120      127.0.0.1      80      0x0011
9      2026-09-11T23:24:49.490513427+0100      127.0.0.1      80      127.0.0.1      51120      0x0011

(ponzhi@kali)~-[~/SBT-DF203-Lab3]
$
```

Part B - Conduct the Bounded Simulation Script and Execution

A Scapy script called `syn_probe_lab.py` was created as instructed in the lab. It sends four SYN packets to 127.0.0.1:80 using random source ports and then stops running. This script only targets the local authorized instance and does not run continuously.



A terminal window titled 'ponzhi@kali: ~/SBT-DF203-Lab3' shows the creation of a script. The user enters `cat > scripts/syn_probe_lab.py <<'PYLAB'`. The script content is displayed: it imports `IP`, `TCP`, `RandShort`, and `send` from `scapy.all`; sets `TARGET = '127.0.0.1'`, `PORT = 80`, and `COUNT = 4`; and contains a loop to create and send SYN packets. A 'Screenshot taken' notification is visible in the top right corner.

```
(ponzhi@kali)-[~/SBT-DF203-Lab3]
$ cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

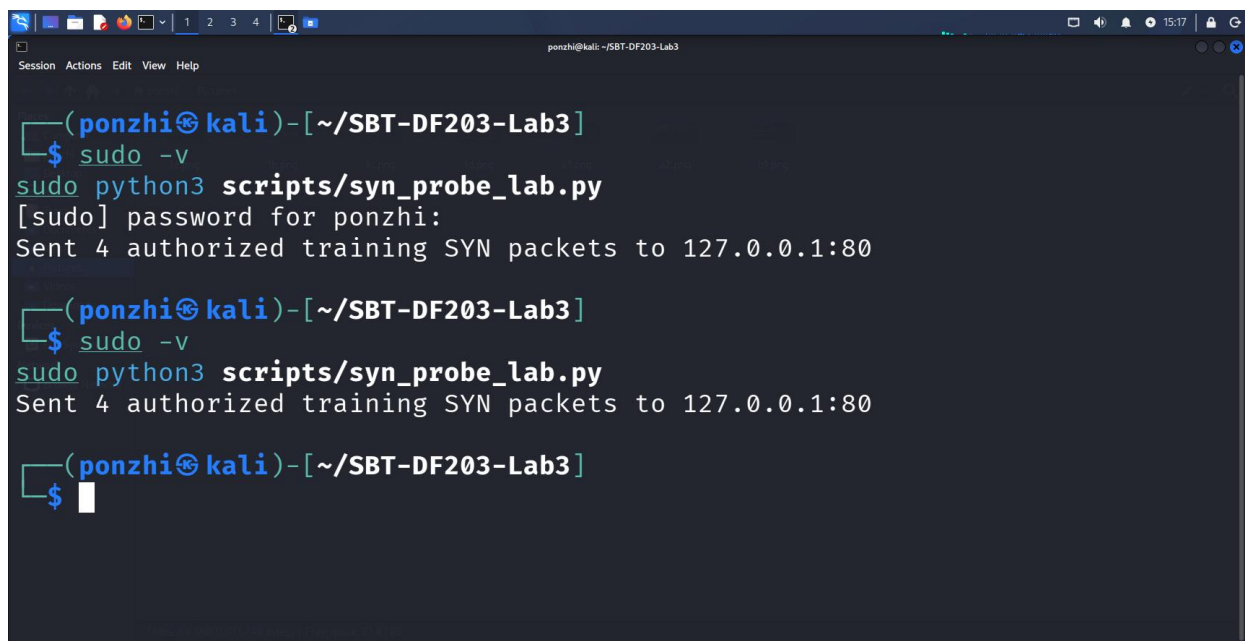
TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB

(ponzhi@kali)-[~/SBT-DF203-Lab3]
$ cat scripts/syn_probe_lab.py
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
```



A terminal window titled 'ponzhi@kali: ~/SBT-DF203-Lab3' shows the execution of the script. The user enters `sudo -v` and `sudo python3 scripts/syn_probe_lab.py`. The output shows the password prompt and the message 'Sent 4 authorized training SYN packets to 127.0.0.1:80'. The process is repeated once more.

```
(ponzhi@kali)-[~/SBT-DF203-Lab3]
$ sudo -v
sudo python3 scripts/syn_probe_lab.py
[sudo] password for ponzhi:
Sent 4 authorized training SYN packets to 127.0.0.1:80

(ponzhi@kali)-[~/SBT-DF203-Lab3]
$ sudo -v
sudo python3 scripts/syn_probe_lab.py
Sent 4 authorized training SYN packets to 127.0.0.1:80

(ponzhi@kali)-[~/SBT-DF203-Lab3]
$
```

Part C – Identify SYN Indicators with tshark

Working Copy and Hash Verification

```
(ponzhi@kali)~[~/SBT-DF203-Lab3]
$ PCAP=working/bounded_syn_activity_working.pcapng

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$ cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$ sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt
e5bbc6fd8689c505b0e521c2129b05a60893814adf14aa64e6a6c632b239dff0  evidence/bounded_syn_activity.pca
png
e5bbc6fd8689c505b0e521c2129b05a60893814adf14aa64e6a6c632b239dff0  working/bounded_syn_activity_work
ing.pcapng

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$
```

SYN Indicator Extraction

```
(ponzhi@kali)~[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq
\
| tee reports/initial_syns.tsv
1      1789308928.458486042      127.0.0.1      2308      127.0.0.1      80      0
4      1789308928.458985042      127.0.0.1      14995     127.0.0.1      80      0
7      1789308928.459356196      127.0.0.1      9856      127.0.0.1      80      0
10     1789308928.459697593      127.0.0.1      12550     127.0.0.1      80      0
13     1789308996.550734827      127.0.0.1      45575     127.0.0.1      80      0
16     1789308996.551301586     127.0.0.1      57732     127.0.0.1      80      0
19     1789308996.551632407     127.0.0.1      65396     127.0.0.1      80      0

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$
```

```
ponzhi@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
| tee reports/syn_ack_responses.tsv
2      1789308928.458513166      127.0.0.1      80      127.0.0.1      2308      1
5      1789308928.458993622      127.0.0.1      80      127.0.0.1      14995     1
8      1789308928.459361960      127.0.0.1      80      127.0.0.1      9856      1
11     1789308928.459702864      127.0.0.1      80      127.0.0.1      12550     1
14     1789308996.550826181      127.0.0.1      80      127.0.0.1      45575     1
17     1789308996.551308961      127.0.0.1      80      127.0.0.1      57732     1
20     1789308996.551660881      127.0.0.1      80      127.0.0.1      65396     1

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$
```

```
ponzhi@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
| tee reports/ack_reset_candidates.tsv
2      1789308928.458513166      127.0.0.1      80      127.0.0.1      2308      0x0012
3      1789308928.458575618      127.0.0.1      2308     127.0.0.1      80        0x0004
5      1789308928.458993622      127.0.0.1      80      127.0.0.1      14995     0x0012
6      1789308928.459018494      127.0.0.1      14995    127.0.0.1      80        0x0004
8      1789308928.459361960      127.0.0.1      80      127.0.0.1      9856      0x0012
9      1789308928.459388834      127.0.0.1      9856     127.0.0.1      80        0x0004
11     1789308928.459702864      127.0.0.1      80      127.0.0.1      12550     0x0012
12     1789308928.459725146      127.0.0.1      12550    127.0.0.1      80        0x0004
14     1789308996.550826181      127.0.0.1      80      127.0.0.1      45575     0x0012
15     1789308996.550867573      127.0.0.1      45575    127.0.0.1      80        0x0004
17     1789308996.551308961      127.0.0.1      80      127.0.0.1      57732     0x0012
18     1789308996.551311907      127.0.0.1      57732    127.0.0.1      80        0x0004
20     1789308996.551660881      127.0.0.1      80      127.0.0.1      65396     0x0012

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$
```

Part D - Quantitative and Correlation of the Activity

```
ponzhi@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
  | tee reports/syn_counts_by_pair.txt
  7 127.0.0.1      127.0.0.1      80
  1 127.0.0.1      127.0.0.1      80

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport \
  | sort -n | uniq | tee reports/unique_syn_source_ports.txt
2308
9856
12550
14995
45575
57732
65396

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$
```

```
ponzhi@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help

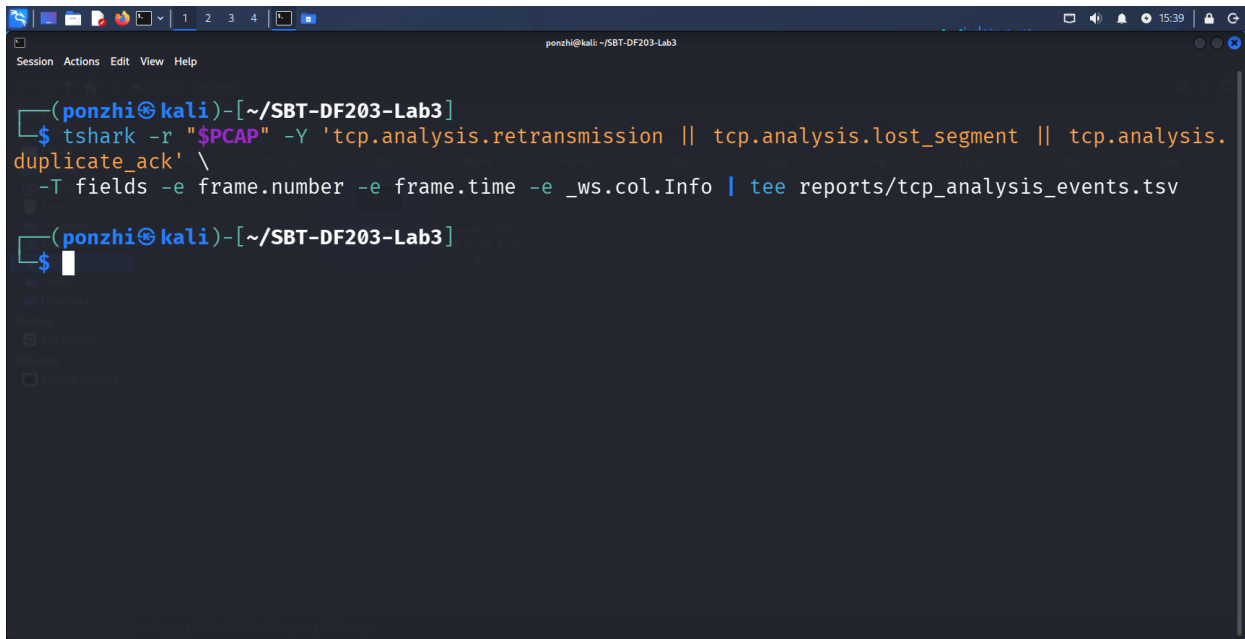
(ponzhi@kali)~[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt

Warns (6)
=====
Frequency  Group      Protocol  Summary
-----
6          Sequence   TCP       Connection reset (RST)

Notes (7)
=====
Frequency  Group      Protocol  Summary
-----
7          Protocol   TCP       The SYN packet does not contain a SACK PERM option

Chats (14)
=====
Frequency  Group      Protocol  Summary
-----
7          Sequence   TCP       Connection establish request (SYN): server port 80
7          Sequence   TCP       Connection establish acknowledge (SYN+ACK): server port
80

(ponzhi@kali)~[~/SBT-DF203-Lab3]
$
```

A terminal window titled 'ponzhi@kali: ~/SBT-DF203-Lab3' with a menu bar (Session, Actions, Edit, View, Help) and a toolbar. The terminal shows a user prompt '(ponzhi@kali)~[~/SBT-DF203-Lab3]' followed by a tshark command: '\$ tshark -r "\$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \ -T fields -e frame.number -e frame.time -e _ws.col.Info | tee reports/tcp_analysis_events.tsv'. The prompt returns, and the user enters a dollar sign '\$' followed by a cursor.

```
(ponzhi@kali)~[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
-T fields -e frame.number -e frame.time -e _ws.col.Info | tee reports/tcp_analysis_events.tsv
(ponzhi@kali)~[~/SBT-DF203-Lab3]
$
```

The expert information summary supports the manual checks: there are 7 SYN packets, 7 SYN-ACK packets, and 6 resets. There are no retransmissions or duplicate ACKs. The few packets and lack of retransmissions indicate that the network capture is clean and not congested, showing no signs of stress.

Part E – Compare Normal and Suspicious Sessions

The table below shows the key numbers we gathered, comparing normal and suspicious cases as required by the lab. It also includes an explanation of what each number means.

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Initial SYN count	1	7	The bounded activity shows multiple independent initial SYNs from a single loopback source across two script runs, versus the single SYN of one ordinary browser/curl request.
SYN-ACK count	1	7	Apache (via the kernel TCP stack) answered every SYN with a SYN-ACK in both cases, showing the service itself was reachable and responsive throughout.
Completed handshakes	1 (SYN → SYN-ACK → ACK; session then closed gracefully via FIN/ACK, frames 8–9)	0	The normal session finished the three-way handshake and exchanged HTTP data before a clean close. None of the bounded-activity sessions were completed by the raw-socket client — the OS TCP stack answered but no application-layer ACK/data followed.
Unique client source ports	1	7	A real client uses one temporary port for each connection. When seven different random source ports appear within a short time frame, it usually indicates that the SYN requests are being generated by a script or spoofed source, rather than by a real user browsing the internet.
HTTP request present?	Yes	No	The regular session involved a real HTTP GET request. In contrast, the bounded packets were just SYN packets without

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
			any application data. This pattern suggests that they were used for scanning or flooding rather than for actual service use.
RST packets	0	6	Six of the seven half-open bounded sessions were reset shortly after the SYN-ACK, because Scapy's raw socket never completed the handshake and the kernel tore the state down. Elevated RSTs following SYN-ACKs are a strong half-open-connection indicator.
Observed duration	15s	Approximately 68s	The packets from the activity group together in two quick bursts, with about a minute in between. This matches the two times the training script was run manually, rather than showing a steady, growing flow.

A real SYN flood attack involves a large scale, a consistent rate, ongoing activity, and noticeable effects on service—none of which apply here. The four-packet training simulation, run twice, imitates the packet patterns that analysts need to recognize. This includes repeated SYN messages that don't get answered, high levels of resets, and randomizing ports for each connection. This simulation is safe and controlled, so it doesn't create an actual denial-of-service situation.

Academic Integrity and Authorization Statement

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.